CNN Practical Implementation

Structured Notes

Cats vs Dogs Classification, GPU

Setup, Data Preparation,
Preprocessing Pipeline, DataLoader, Convolution on Real Images

Contents

1 Introduction and Problem Overview **2**
1.1 Mathematical Formulation . . . 2
1.2 What CNN Learns . . . 2
1.3 Concept Check . . . 2

2 Environment Setup: GPU vs CPU **2**
2.1 Device Selection in PyTorch . . . 3
2.2 GPU vs CPU Benchmark . . . 3
2.3 TPU vs GPU . . . 3
2.4 Concept Check . . . 3

3 Dataset Structure and Data Preparation **4**
3.1 Cats vs Dogs Dataset . . . 4
3.2 Loading Image Paths . . . 4
3.3 Extracting Labels from File Names . . . 4
3.4 Label Encoding . . . 4
3.5 Creating a Structured DataFrame . . . 4

4 Train, Validation, and Test Split **5**
4.1 Stratified Sampling . . . 5

5 Image Preprocessing Pipeline **6**
5.1 Why Preprocessing is Necessary . . . 6
5.2 Preprocessing Steps . . . 6
5.3 Resizing: Trade-offs . . . 6
5.4 Normalization . . . 6
5.5 Dimension Rearrangement . . . 7
5.6 Saving Preprocessed Tensors as .pt Files . . . 7
5.7 Beyond Basic Preprocessing . . . 7

6 DataLoader and Batching **8**
6.1 Why Use DataLoader? . . . 8
6.2 PyTorch DataLoader . . . 8
6.3 Key DataLoader Parameters . . . 9
6.4 What the Model Sees . . . 9
6.5 Moving Data to Device . . . 9
6.6 Batch Size Trade-offs . . . 9

7 Convolution on Real Images **9**
7.1 How Filters Work . . . 9
7.2 Types of Filters . . . 10
7.3 Key Insight: CNN Learns Filters Automatically . . . 10

8 Pooling on Real Images **11**
8.1 Max Pooling . . . 11

9 Complete Deep Learning Pipeline Overview **11**

10 Summary **11**

1 Introduction and Problem Overview

CNN Image Classification

Convolutional Neural Networks (CNNs) can take an image as input, learn patterns through convolutional filters, and classify images into categories. In this implementation, the goal is to classify images as either **cat** or **dog** using real-world image data.

Previously, convolution, pooling, and CNN architectures were studied using dummy matrices and simple filters. The critical next step is applying these concepts to **real-world images** with real noise, varying lighting, different angles, and complex backgrounds.

1.1 Mathematical Formulation

An image can be represented as a 3D tensor:

$$X \in \mathbb{R}^{H \times W \times C}$$

where H is height, W is width, and C is the number of channels (3 for RGB).

The model objective is to learn a function:

$$f(X; \theta) \rightarrow \{0, 1\}$$

where $Y = 0$ for cat and $Y = 1$ for dog, X is the input image, and θ represents the CNN parameters (filters and weights).

1.2 What CNN Learns

Instead of manually defining features, CNN learns edges, textures, and object shapes automatically. The learning process follows:

1. **Prediction**: Forward pass through the network
2. **Loss**: Compute the difference between prediction and ground truth
3. **Backpropagation**: Calculate gradients
4. **Update weights**: Adjust parameters to reduce loss

This is the same process as in ANN, but now applied to images instead of flat vectors.

1.3 Concept Check

- **Why can't we directly feed image files?** CNN operates on numerical tensors. Images must be converted into arrays of pixels before processing.
- **What is the shape of a color image?** Height × Width × Channel.
- **What does CNN learn from images?** Hierarchical features: edges, textures, shapes, and object-level representations.
- **ANN input vs CNN input**: ANN uses flat vector input; CNN uses structured spatial input.
- **Why validation and test sets?** Validation for tuning model hyperparameters; test for final unbiased evaluation of model performance.

2 Environment Setup: GPU vs CPU

GPU for Deep Learning

Deep learning involves large matrix operations, convolution across images, and millions of parameters. This is computationally expensive. **GPU** (Graphics Processing Unit) processes operations

in parallel, making it significantly faster than CPU for deep learning tasks.

Aspect	CPU	GPU
Processing	Sequential	Parallel
Best for	General tasks	Deep learning, matrix operations
Speed (large data)	Slower	Much faster
Initial latency	None	Data transfer overhead

2.1 Device Selection in PyTorch

Device Selection

```python
import torch

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")
```

Listing 1: Setting up computation device

Critical Rule

Every tensor and model must be moved to the **same device**. If the model is on GPU but data is on CPU, PyTorch will throw a `RuntimeError` due to device mismatch. Think of CPU as a normal worker and GPU as thousands of workers operating in parallel.

2.2 GPU vs CPU Benchmark

For a single small matrix multiplication (size 1000), CPU can be faster than GPU. This is because GPU has an **initial latency**: data must first be transferred to the GPU hardware before processing begins. CPU starts processing immediately.

However, for repeated operations (e.g., size 4000, 20 repetitions), GPU is significantly faster. In CNN training, data is processed in batches repeatedly, making GPU the standard choice.

Key insight:

- Single task → CPU can be faster (no transfer overhead)
- Repeated matrix operations (CNN training) → GPU is much faster (parallel processing)
- GPU is standard for training; CPU is often used only for inference

2.3 TPU vs GPU

TPU (Tensor Processing Unit) is designed specifically for tensor operations and is above GPU in capability. GPU stands for Graphics Processing Unit and was originally designed for graphics but is widely used for deep learning. TPU is specialized for deep learning workloads.

2.4 Concept Check

- **Why is GPU preferred for deep learning?** GPU performs parallel computation, ideal for matrix operations and convolutions.
- **What happens if model is on GPU but data is on CPU?** Runtime error due to device mismatch.
- **What does `torch.cuda.is_available()` check?** Whether a CUDA-enabled GPU is available for computation.
- **Why use `.to(device)`?** To move tensors and models to the same computation device.

- **Is GPU always required?** No, but it significantly speeds up training, especially for large datasets and deep models.

3 Dataset Structure and Data Preparation

3.1 Cats vs Dogs Dataset

The dataset from Kaggle contains 25,000 images total. Images are stored as files with labels embedded in the file names (e.g., `cat.0.jpg`, `dog.1.jpg`). There is no structured format like CSV—this is **unstructured data** that must be converted into a structured format.

3.2 Loading Image Paths

Loading Image Paths

```
import os
import glob

dataset_path = "path/to/training_images"
image_paths = glob.glob(os.path.join(dataset_path, "*.jpg"))
print(f"Total images found: {len(image_paths)}")
# Output: Total images found: 25000
```

Listing 2: Collecting all image file paths

3.3 Extracting Labels from File Names

Each file is named like `cat.0.jpg` or `dog.1234.jpg`. The label is extracted using string splitting:

Label Extraction

```
# filename: cat.0.jpg
# Split by '.' -> ['cat', '0', 'jpg']
# First element is the label

labels = []
for path in image_paths:
    filename = os.path.basename(path)
    label = filename.split('.')[0]  # 'cat' or 'dog'
    labels.append(label)
```

Listing 3: Extracting labels from filenames

3.4 Label Encoding

After extracting text labels, they are converted to numeric values:

- `cat` → 0
- `dog` → 1

This is **label encoding**. For binary classification, 0 and 1 suffice. For multi-class problems, **one-hot encoding** would be used instead.

3.5 Creating a Structured DataFrame

The unstructured image files are converted into a structured tabular DataFrame with columns: `file_path`, `label` (text), and `label` (numeric).

DataFrame Creation

```python
import pandas as pd

df = pd.DataFrame({
    'file_path': image_paths,
    'label_text': labels,
    'label': [0 if l == 'cat' else 1 for l in labels]
})
print(f"Cat images: {(df['label'] == 0).sum()}")
# Cat images: 12500
print(f"Dog images: {(df['label'] == 1).sum()}")
# Dog images: 12500
```

Listing 4: Creating structured DataFrame

This step matters because now the data can be shuffled, split into train/test/validation, and preprocessing can be applied systematically. This is exactly how a real ML pipeline starts: raw data → structured format → preprocessing → model.

4 Train, Validation, and Test Split

Data Splitting

The dataset is divided into three non-overlapping subsets:

- **Training set (70%)**: Used to train the model
- **Validation set (15%)**: Used for tuning hyperparameters
- **Test set (15%)**: Used for final, unbiased evaluation

Mathematical representation:

$$D = D_{\text{train}} \cup D_{\text{val}} \cup D_{\text{test}}, \quad D_{\text{train}} \cap D_{\text{val}} \cap D_{\text{test}} = \emptyset$$

Important rule: Test data must **never** be used during training.

Data Splitting

```python
from sklearn.model_selection import train_test_split

X_train, X_temp, y_train, y_temp = train_test_split(
    df['file_path'], df['label'],
    test_size=0.3, random_state=42, stratify=df['label']
)
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp,
    test_size=0.5, random_state=42, stratify=y_temp
)
# Training: 8400, Validation: 1800, Test: 1800
```

Listing 5: Splitting data with stratification

4.1 Stratified Sampling

Stratified sampling ensures that the class distribution (cat vs dog) is preserved in each split. The training, validation, and test sets each contain approximately equal numbers of cat and dog images (≈4200 cats, ≈4200 dogs in training).

K-Fold Cross-Validation

For more robust results, K-fold cross-validation can be applied with different training and test splits to determine on which data partition the model performs best.

5 Image Preprocessing Pipeline

5.1 Why Preprocessing is Necessary

Raw images cannot go directly into CNN due to:

- **Different sizes**: Images have varying dimensions (e.g., 500×444, 375×500)
- **Different lighting conditions**: Brightness and contrast vary
- **Different pixel ranges**: Raw pixel values range from 0 to 255

CNN works on a **fixed dimension**. If dimensions keep varying, the filters cannot operate consistently and a size mismatch error will occur. The CNN layers are not dynamic—they are fixed with size.

5.2 Preprocessing Steps

1. **Resize** images to fixed dimensions (128×128)
2. **Convert** BGR to RGB (OpenCV loads in BGR format)
3. **Normalize** pixel values from $[0, 255]$ to $[0, 1]$
4. **Convert** to tensor format

5.3 Resizing: Trade-offs

Factor	Smaller Images (128)	Larger Images (224)
Training speed	Faster	Slower
Feature capture	Less detail	Better detail
Compute required	Less	More
Information loss	More	Less

When resizing, there is always some **loss of information**. Reducing image quality loses detail; enhancing quality adds noise. This is a trade-off between speed and accuracy. The image size is a **hyperparameter to tune** based on available compute.

5.4 Normalization

Simple normalization:

$$X_{\text{norm}} = \frac{X}{255}$$

This scales pixel values from $[0, 255]$ to $[0, 1]$.

Mean-Standard normalization (used with pre-trained models like ResNet):

$$X_{\text{norm}} = \frac{X - \mu}{\sigma}$$

where μ is the mean and σ is the standard deviation of the dataset.

Why normalize? Faster convergence, stable gradients, and better training.

5.5 Dimension Rearrangement

OpenCV loads images in the format: **Height** × **Width** × **Channel** (HWC). PyTorch expects: **Channel** × **Height** × **Width** (CHW).

Image to Tensor Conversion

```python
import cv2
import numpy as np
import torch

def preprocess_image(path, img_size=128):
    img = cv2.imread(path)
    img = cv2.resize(img, (img_size, img_size))
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    img = img / 255.0  # Normalize to [0, 1]
    # Rearrange: HWC -> CHW
    img = np.transpose(img, (2, 0, 1))
    return torch.tensor(img, dtype=torch.float32)
```

Listing 6: Converting image to PyTorch tensor

5.6 Saving Preprocessed Tensors as .pt Files

Saving Tensors

```python
torch.save((X_train, y_train), "X_train.pt")
torch.save((X_val, y_val), "X_val.pt")
torch.save((X_test, y_test), "X_test.pt")
```

Listing 7: Saving preprocessed data

Why Save as .pt Files?

Once preprocessed data is saved as `.pt` files, the preprocessing step does not need to be repeated. The tensors can be loaded directly with `torch.load()`, enabling faster experimentation. The `.pt` file contains tensor information plus label information—the complete DataFrame in tensor format. This is why the file size increases (e.g., 525 MB of images becomes ≈1.5 GB as tensors with metadata). An alternative format is `.pkl` (pickle).

5.7 Beyond Basic Preprocessing

The basic pipeline (resize, normalize, tensor) has limitations:

- Fixed resizing may distort aspect ratio
- No data augmentation—model may overfit
- Normalization is generic (0 to 1 scaling)

Data augmentation creates variations without increasing data manually: horizontal flip, rotation, zoom, brightness change. More data diversity leads to better generalization.

On-the-fly preprocessing uses custom dataset classes to load and transform images dynamically instead of saving `.pt` files. This requires more computation but is more flexible.

Key Takeaway

In real ML projects, preprocessing is not fixed—it is a **design decision** based on the data, compute resources, and model requirements. What is built initially is a baseline pipeline; industry uses

refined and optimized pipelines.

6 DataLoader and Batching

6.1 Why Use DataLoader?

Feeding the entire dataset at once is:

- Memory inefficient
- Computationally expensive
- Not scalable

The solution is **mini-batches**. Instead of the full dataset, data is fed in batches.

Example calculation:

- Dataset size: 8,400 images
- Batch size: 32
- Total batches: $\lceil 8400/32 \rceil = 263$

Benefits of mini-batch training: faster training, stable gradient updates, and better generalization.

6.2 PyTorch DataLoader

DataLoader Setup

```python
from torch.utils.data import TensorDataset, DataLoader

# Load saved tensors
X_train, y_train = torch.load("X_train.pt")
X_val, y_val = torch.load("X_val.pt")
X_test, y_test = torch.load("X_test.pt")

# Create datasets
train_dataset = TensorDataset(X_train, y_train)
val_dataset = TensorDataset(X_val, y_val)
test_dataset = TensorDataset(X_test, y_test)

# Create DataLoaders
batch_size = 32
train_loader = DataLoader(train_dataset, batch_size=batch_size,
                          shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=batch_size,
                        shuffle=False)
test_loader = DataLoader(test_dataset, batch_size=batch_size,
                         shuffle=False)
```

Listing 8: Creating DataLoaders

Shuffle Rules

Training loader: `shuffle=True` — randomizes data order each epoch for better generalization.

Validation and test loaders: `shuffle=False` — consistent ordering for reproducible evaluation.

6.3 Key DataLoader Parameters

Parameter	Description
`batch_size`	Number of samples per batch
`shuffle`	If True, randomizes the data each epoch
`num_workers`	Number of parallel data loading processes

6.4 What the Model Sees

The model never sees the full dataset. It processes one batch at a time:

Batch tensor shape:

$$\text{Batch} \in \mathbb{R}^{B \times C \times H \times W}$$

where B = batch size, C = channels, H = height, W = width.

Example: batch size 32, image size 128×128 ⇒ shape is $[32, 3, 128, 128]$.

6.5 Moving Data to Device

Data and model must be on the same device to avoid runtime errors:

Moving to GPU

```
for images, labels in train_loader:
    images = images.to(device)  # Move to GPU
    labels = labels.to(device)  # Move to GPU
    break

print(f"Device of images: {images.device}")
# Device of images: cuda:0
```

Listing 9: Transferring batch to GPU

6.6 Batch Size Trade-offs

- **Very small batch size**: High noise in gradients, training is unstable
- **Very large batch size**: Less noise, more stable, but may hurt generalization
- A **balanced batch size** is needed—batch size is an important hyperparameter

Mini-Batch Gradient Descent

Batching introduces noise in gradients, which leads to **better generalization** and **faster convergence**. This is why mini-batch gradient descent is used instead of full-batch or stochastic gradient descent.

7 Convolution on Real Images

Previously, convolution was studied using dummy matrices and simple filters. The real question is: what happens when convolution is applied to **real images**?

CNN does not see objects—it detects **edges**, **patterns**, and **textures**.

7.1 How Filters Work

A filter (kernel) slides over the image. The output at each position is the dot product of image pixels and kernel weights:

$$\text{Output}(i,j) = \sum_m \sum_n \text{Image}(i+m, j+n) \cdot \text{Kernel}(m,n)$$

The filter highlights specific patterns and suppresses irrelevant information.

7.2 Types of Filters

Filter Type	What It Does	Use Case
Edge Detection	Highlights boundaries, detects shapes	Finding object boundaries
Sharpen	Enhances details, increases contrast	Improving image clarity
Blur	Smooths images, removes noise	Noise reduction

These filters are available in OpenCV and can be applied using `cv2.filter2D()`.

Applying Filters

```python
import cv2
import numpy as np

# Convert image to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Edge detection filter
edge_filter = np.array([[-1, -1, -1],
                        [-1,  8, -1],
                        [-1, -1, -1]])

# Sharpen filter
sharpen_filter = np.array([[ 0, -1,  0],
                           [-1,  5, -1],
                           [ 0, -1,  0]])

# Blur filter
blur_filter = np.ones((3, 3)) / 9.0

# Apply filters
edge_img = cv2.filter2D(gray, -1, edge_filter)
sharp_img = cv2.filter2D(gray, -1, sharpen_filter)
blur_img = cv2.filter2D(gray, -1, blur_filter)
```

Listing 10: Applying convolution filters on real images

7.3 Key Insight: CNN Learns Filters Automatically

Each filter extracts a different feature from the image. In CNN, instead of manually defining these filters, the network **learns the filter weights automatically** through backpropagation.

Filter Behavior During Training

- The **kernel shape** is fixed once decided and does not change during training
- Only the **weights** within the filter change during training
- The **nature** of a filter does not change—if a filter detects edges, it will not start behaving like

a sharpen or blur filter
- What changes is the **precision**: the filter becomes better at what it is designed to detect
- In a pre-trained model, filters, layers, and weights are already defined and used in the same hierarchical order

8 Pooling on Real Images

8.1 Max Pooling

Max pooling reduces the spatial dimensions by selecting the **maximum value** from each patch of the feature map.

For a 2×2 max pooling with stride 2:

$$\text{Output}(i,j) = \max\left(\text{Input}(2i,2j),\ \text{Input}(2i+1,2j),\ \text{Input}(2i,2j+1),\ \text{Input}(2i+1,2j+1)\right)$$

When applied to a real image:

- The image size is reduced
- Important features become more **prominent**
- Redundant information is removed
- Only the best features are preserved from each patch

Pooling Purpose

Max pooling keeps important signals and removes redundant information. It reduces the spatial dimensions while preserving the most significant features, making the representation more compact and computation more efficient.

9 Complete Deep Learning Pipeline Overview

The end-to-end pipeline for CNN image classification follows these steps:

1. **Data Preparation**: Load images, create structured DataFrame, split into train/validation/test
2. **Preprocessing**: Resize images, convert BGR to RGB, normalize pixel values, convert to tensors, save as `.pt` files
3. **CNN from Scratch**: Build a custom CNN, train the model, evaluate performance
4. **Visual Understanding**: Apply convolution filters on real images, visualize feature maps, understand pooling effect
5. **Transfer Learning with ResNet-18**: Use a pre-trained model, modify the final layer, train efficiently
6. **Model Comparison**: Compare accuracy, training time, and generalization ability
7. **Final Prediction**: Test on unseen images, interpret model outputs

This pipeline is exactly how real-world computer vision systems are built.

10 Summary

- Images are represented as 3D tensors ($H \times W \times C$) and CNN learns a mapping function $f(X;\theta) \to \{0,1\}$ for binary classification
- GPU is preferred for deep learning due to parallel computation; all tensors and models must be on the **same device**
- Unstructured image files are converted to structured DataFrames with file paths and labels for systematic processing
- Data is split into training (70%), validation (15%), and test (15%) with **stratified sampling** to

preserve class distribution
- Preprocessing involves resizing to fixed dimensions, BGR to RGB conversion, normalization ($[0, 255] \rightarrow [0, 1]$), and HWC to CHW rearrangement
- Preprocessed tensors are saved as `.pt` files to avoid repeated preprocessing during model experimentation
- DataLoader provides efficient mini-batch loading with shuffling for training and consistent ordering for evaluation
- CNN filters (edge detection, sharpen, blur) extract different features from images; CNN learns these filters automatically through backpropagation
- Filter kernel shape is fixed during training; only weights change, improving the precision of feature detection
- Max pooling reduces spatial dimensions while preserving the most significant features
- Preprocessing is a design decision—image size is a trade-off between speed and accuracy based on available compute